HATEOAS generally introduces more complexity than is worthwhile for small projects, but it's a great way to decouple your client-side applications from your server APIs so that they can evolve separately. This approach can be invaluable when you have large or independent teams. The problem for OpenAPI is that it wasn't designed for these kinds of dynamic APIs. OpenAPI wants to know up front what the responses are for each of your endpoints, which isn't information that you can give it if you're following HATEOAS.

In a different scenario, you may have multiple backend APIs, each with its own OpenAPI specification. You expose a single, unified API gateway app, with which all your clients interact. Unfortunately, even though each backend API has an OpenAPI specification, there's no easy way to combine the APIs into a single unified document that you can expose in your API gateway and which clients can use for testing and code generation.

Another common problem centers on securing your APIs with authentication and authorization. The OpenAPI specification contains a section about describing your authentication requirements, and Swagger UI supports them. Where things fall down is if you're using any extensions to the common authentication protocols or advanced features. Although some of these workflows are possible, in some cases Swagger UI simply may not support your workflow, rendering Swagger UI unusable.

## *11.6.2 Generated code is opinionated*

At the end of section 11.4 I said that code generation is the killer feature for Open API documents, and in many cases it is. That statement, however, assumes that you *like* the generated code. If the tooling you use—whether that's NSwag or some other code generator—doesn't generate the code you want, you may find yourself spending a lot of effort customizing and tweaking the output. At some point and for some APIs, it may be simpler and easier to write your own client!